

The Limitless Stack

Seven tools, one operating system.
Built by you, for you — on your own accounts.





Welcome

What you'll build, and how to use this guide

The Limitless Stack is seven tools wired into one operating system for working with a coding agent: **Claude · CLAUDE.md · Obsidian · NotebookLM · Pinecone · Hub Workspace · Paperclip**. It gives your agent a memory it reads *and* writes, rules it follows every session, and rituals that make it smarter every time you use it.

This guide shows you how to build **your own** copy. You get the same structure, the same tools and the same rituals as the reference stack. Everything inside it is yours: your vault, your notes, your notebooks, your index, your rules. Nothing is shared with anyone else's stack, and nothing here points at anyone else's accounts. (A team can also keep one shared vault; section 08 shows how.)

Hand this PDF to your coding agent. Claude Code, the Claude desktop app with a folder on your computer connected, or any other coding agent that can read files and run commands on your computer. Attach this file and say: "Set up my own Limitless Stack by following this guide. Ask me for my details first." Your agent interviews you, runs the commands on your computer, and checks each step before moving on. You can also follow it by hand. **Using Grok Bot?** Set it up with section 09 first.

Your details — what your agent will ask you first

Detail	Used for	Example
Your name	The [YOUR NAME] markers in your CLAUDE.md	Sam
Your GitHub account	Where your vault's private repo lives	sam-rivera
A short project ID	Your vault, notebook names and Pinecone index	sam-research
Where your vault lives	The folder everything is built in	~/sam-research
Your domain	What the wiki tracks	"my consulting clients"
Your Google account	Your NotebookLM notebooks	the account you sign in with
Pinecone (optional)	Semantic search over your files	a free account + API key
How you want to work	The working rules in your CLAUDE.md	"always ask before pushing"

By the end you'll have

- A **wiki vault** in Obsidian that your agent maintains: every source you add updates many pages.
- **Your own NotebookLM notebooks**, including a reminder notebook your agent reads at the start of every session.
- **Mechanical rituals**: Roll Call checks the stack before work; an end-of-session checklist leaves it clean.

- An **anti-patterns ledger** — your agent's record of mistakes not to repeat — seeded with twelve starter lessons.
- **Seven skills** for Claude (the same steps are written out for any other agent), and optional semantic search (Pinecone) over your own files.

01 The seven tools

Each plays one role. The protocol is how they connect.

Tool	Role	In your stack
Claude	Reasoning engine	Claude or any coding agent you use: reads, writes, connects, decides
CLAUDE.md	Identity + rules	Your vault's operating manual — the trust anchor. Required.
Obsidian	Knowledge base	Your wiki: sources, entities, concepts, syntheses. Required.
NotebookLM	Research desk + reminder layer	Your notebooks, declared in your manifest. Required.
Pinecone	Semantic memory	Your index over your files and repos. Optional.
Hub Workspace	Shared agent workspace	For teams that run a Hub. Optional.
Paperclip	Agent coordination	Org chart, budgets, tickets for agents. Optional.

The three problems it solves

Amnesia. Every agent session starts cold, and long contexts get expensive and unreliable. The stack gives your agent a persistent memory it can read and write — the wiki, NotebookLM and (optionally) Pinecone — so knowledge compounds instead of evaporating.

Drift. Rules and lessons written in files get ignored unless something enforces them. The stack enforces them three ways: a mandatory first-action banner in CLAUDE.md, skills that trigger on your greeting, and a Roll Call script that mechanically gates work.

Repair at scale. Once you run several apps, manual bug triage doesn't scale. The optional self-healing loop diagnoses bugs, drafts fixes in a sandbox and opens pull requests for you to review.

02 The flywheel

Four loops that make the stack smarter every time you use it

Loop 1 — Source ingest. Drop a source in raw/. Your agent reads it, writes a source page, updates every entity and concept it touches, and flags contradictions. The more you ingest, the more cross-references the next ingest can make.

Loop 2 — Wiki refinement. Contradictions are flagged, never silently overwritten. Good answers are filed back as synthesis pages. A periodic lint catches orphans, broken links and stale claims.

Loop 3 — Index sync. The end-of-session checklist refreshes your NotebookLM notebooks and **verifies the content landed**, and syncs Pinecone if you use it. Every session's wiki edits reach your notebooks before the session closes.

Loop 4 — Anti-patterns + reminder notebook. When your agent repeats a mistake, it becomes a numbered entry in your anti-patterns page. That page lives in your reminder notebook, so the next session reads the lesson before doing anything.

03 What compounding looks like

Measured numbers from the reference vault

These are the real counts from the vault the stack was built in, read from its git history on 2026-09-23. Your numbers will differ — they depend on how much you ingest — but the shape is the point: pages and lessons keep accumulating, session after session.

	Your day 1	Day ~30	Day ~90	Day 161
Wiki pages	8 starter pages	81	104	160
Anti-patterns	12 starter lessons	16	29	79
NotebookLM notebooks	2 (yours)			8 routed
Pinecone vectors	0 (optional)			34,506

If your numbers stop moving, the loop has a leak — almost always a skipped end-of-session step. Roll Call exists to catch those leaks at the start of the next session.

04 Self-healing (optional)

Autonomous bug repair, with you approving every merge

Once your stack is running, you can let Claude repair bugs in your own apps. It is opt-in and off by default: you add it to one app repo at a time from the templates in `self-heal/templates/` (the installer also copies them into `self-heal-templates/` in your vault).

#	Stage	What happens
1	Capture	A user reports a bug in your app; description + context are posted to your bug-report endpoint.
2	Diagnose	Claude, with your <code>CLAUDE.md</code> as its rules, writes a diagnosis: severity, root cause, suspected files.
3	Review	You read the diagnosis and decide whether to dispatch a repair.
4	Dispatch	A GitHub <code>repository_dispatch</code> event starts the repair workflow.
5	Init	A GitHub Actions runner checks out a branch. Sandboxed — never production.
6	Investigate	The agent works with a fixed tool list and a 25-turn budget.
7	Commit	If it changed code it commits; if not, it reports why.
8	PR	It opens a pull request with the full context.
9	Merge	You review. Auto-merge is off by default.
10	Reconcile	The bug record is updated; new failure patterns go into your anti-patterns page.

Five rules that make it trustworthy

- **CLAUDE.md is the trust anchor** — both agents run with it as their rules.
- **Diagnose before repair** — the cheap pass always runs first; repair is gated by you.
- **Sandboxed execution** — an ephemeral runner with a code-enforced tool list.
- **Human merge by default** — pull requests, never direct commits.
- **Closed-loop audit** — every transition is recorded on the bug record.

05

Seven skills, plus the tools

Installed by `install.sh`

Skills are Claude's format: a Markdown file Claude loads when its description matches what you're doing. The installer puts all seven in `~/ .claude/skills/` (read by Claude Code). In the Claude desktop app, add skills under **Settings, Capabilities**. Other agents don't load skills; everything the skills do is also written out in your `CLAUDE.md` and the tools.

Skill	What it does	When it triggers
roll-call	Runs <code>tools/limitless-preflight.sh</code> in your vault and reports READY / WARN / BLOCK.	"hey claude", "roll call", "are you ready", session start
limitless-stack	The umbrella protocol: lookup order, first action, end-of-session checklist.	Work in a Limitless Stack vault
notebooklm	Full NotebookLM CLI: notebooks, sources, questions, generated artifacts.	Any NotebookLM operation
audit-before-claim	Verify before stating: done, fixed, counts, "unavailable".	About to claim a result or report status
karpthy-guidelines	Code-edit discipline: think first, keep it simple, surgical changes.	Writing or reviewing code
impeccable	Frontend design: shape, critique, audit and polish interfaces.	Designing or improving a UI
of-module-hardening	A hardening checklist written for one specific app codebase.	Only that codebase — safe to ignore

The tools in your vault's `tools/` folder

Tool	Purpose
limitless-preflight.sh	Roll Call itself: checks the stack, exit 0 / 1 / 2
session-bootstrap.sh	Session-start snapshot: wiki state, recent log entries, open items
notebooklm-wiki-refresh.py	Mirrors wiki pages to your notebooks and verifies they landed
notebooklm-dedupe.py	Finds and removes duplicate notebook sources
recall.sh	Searches your decision history before you call something missing
pinecone-sync.py / -search.py	Index and search your files (optional)
nightly-selfheal.sh	Optional nightly run: Roll Call, then only the safe automatic fixes (notebook refresh, Pinecone sync), then re-check
authorship-guard.py	Shared vaults only: stops a commit that changes someone else's writing (section 08)

06 Enforcement

Three layers that make the rituals reliable

Layer 1 — the banner at the top of your CLAUDE.md. Claude reads CLAUDE.md automatically when a session opens in your vault; your vault's AGENTS.md points every other coding agent to the same file. It starts with:

```
MANDATORY FIRST ACTION - NO EXCEPTIONS
1. Invoke the roll-call skill. (0 = READY, 1 = WARN, 2 = BLOCK)
2. Run bash tools/session-bootstrap.sh
3. Query this vault's reminder notebook (through the notebooklm skill)
4. Read wiki/index.md
5. Do NOT skip these steps
6. Do NOT answer from active context alone. Verify and cite.
```

Layer 2 — skills that trigger on your greeting (Claude). The roll-call skill lists phrases like "hey claude" and "roll call", so saying hello starts the check even if the banner were missed. With other agents, Layer 3 does this job.

Layer 3 — an explicit prompt, when you want to be sure:

```
Run roll call, then the session bootstrap, then ask my reminder notebook
for my current rules and recent lessons. Then read wiki/index.md.
```

The end of a session works the same way. The most reliable phrase is *"Run the end-of-session checklist."* The two rituals check each other: if a wrap-up was skipped, the next Roll Call flags the uncommitted work.

07 Build your stack

Each step ends with a check. Replace every <...> with your details.

You need: a Mac with Homebrew, Python 3.11 (brew install python@3.11), git and the GitHub CLI (gh auth login), Obsidian, a coding agent (Claude or another agent that can run commands on your computer), and a Google account. Pinecone is optional.

1 Get the Limitless Stack

```
git clone https://github.com/Open-Scaffold-Labs/LimitlessStack.git ~/LimitlessStack
echo 'export LIMITLESS_STACK_HOME=~/.LimitlessStack' >> ~/.zshrc && source ~/.zshrc
```

Check: ls ~/LimitlessStack shows bin/, tools/, skills/, templates/ and install.sh.

2 Create your vault

This writes your vault, your CLAUDE.md, your wiki starter pages and your manifest (`.limitless-project.py`) — the one file that names your notebooks and index.

```
~/LimitlessStack/bin/limitless-stack-init <project-id> <vault-path> --description "<your domain>"
```

Check: the vault folder contains CLAUDE.md, AGENTS.md, `.limitless-project.py`, `tools/` and `wiki/`.

3 Install the skills and tools

```
~/LimitlessStack/install.sh <vault-path>
```

It installs the Python packages and the browser used for NotebookLM sign-in, the seven skills, and the tools. It prints two commands for the optional nightly job; nothing runs on a schedule unless you run them.

Check: `ls ~/ .claude/skills/` lists seven skills.

4 Create your NotebookLM notebooks

```
notebooklm login # your Google account, in a browser, once
notebooklm create "<project-id>"
notebooklm create "<project-id> Reminder"
```

Copy each returned ID into `<vault-path>/ .limitless-project.py`: the first replaces `REPLACE_WITH_NEW_NOTEBOOK_ID`, the second `REPLACE_WITH_REMINDER_NOTEBOOK_ID`. Then fill them:

```
cd <vault-path>
python3.11 tools/notebooklm-wiki-refresh.py --seed
python3.11 tools/notebooklm-wiki-refresh.py
```

On brand-new notebooks the seed warns that two curated files have no match yet. That is expected: the refresh right after it adds them.

Check: the refresh reports `verify_failed: 0` and `upload_failed: 0`.

5 Make CLAUDE.md yours

Your CLAUDE.md has `[YOUR ...]` markers: your name, your domain, your working rules, your notes. Let your agent interview you and fill every one, then write `wiki/overview.md` — your current thesis, in your own words.

Check: `grep -c "\[YOUR" CLAUDE.md` prints 0.

6 Put your vault on GitHub

```
cd <vault-path> && git init && git add -A && git commit -m "initial commit"
gh repo create <your-github-account>/<project-id> --private --source . --push
```

Check: `git status` is clean and the repo shows on GitHub.

7 Pinecone (optional)

Create a free account at pinecone.io and an API key. Store the key in your Keychain, then create an index with Pinecone's hosted embedding model:

```
security add-generic-password -a pinecone -s pinecone-api-key -U -w # paste the key
python3.11 -c 'import subprocess; from pinecone import Pinecone; \
    k=subprocess.run(["security", "find-generic-password", "-s", "pinecone-api-key", "-w"], \
        capture_output=True, text=True, check=True).stdout.strip(); \
    Pinecone(api_key=k).create_index_for_model(name="<project-id>", cloud="aws", \
        region="us-east-1", embed={"model": "multilingual-e5-large", \
        "field_map": {"text": "chunk_text"}})'
```

Your manifest already names the index after your project ID. Add "pinecone" to CHECKS in it, put any repos you want searchable in `raw/repos/`, then run `python3.11 tools/pinecone-sync.py`. The free plan caps embedding at 5 million tokens a month; if the sync reports that limit, wait for the monthly reset or upgrade.

Check: `python3.11 tools/pinecone-search.py "a test question"` returns hits.

8 Run Roll Call

Open your agent in your vault and say "roll call" (or run `bash tools/limitless-preflight.sh`). Work through anything it reports until it says **READY**, or **WARN** with only warnings you choose to accept.

Your first Roll Call will not be green, and that's expected. Until you've signed in to NotebookLM it reports BLOCK. Until your notebooks are created and refreshed, and your overview is written, it lists those as warnings. Every one maps to a step above. Your vault's `wiki/team-tasks.md` holds the same checklist so your agent can tick it off.

08 Operating

Day-to-day moves that keep the loop alive

Start every session. Say "roll call" (Claude also answers to "hey claude"). Your agent checks the stack, reads your reminder notebook and the index.

Ingest a source. Drop it in `raw/` and say "Ingest the new source at `raw/<file>`".

Ask a question. Just ask. Your agent follows the lookup order and cites your wiki. Good answers get filed as synthesis pages.

Lint. Every few weeks: "Run a lint pass and write the report."

Catch a new mistake. Ask your agent to add it to `wiki/synthesis/claude-anti-patterns.md` as the next numbered entry.

End every session. "Run the end-of-session checklist." Tasks ticked, log appended, repos pushed, notebooks refreshed and verified.

Get improvements. `git -C ~/LimitlessStack pull`, then re-run `install.sh` on your vault. It refreshes tools and skills; your `CLAUDE.md`, manifest and wiki are never overwritten.

Make it yours — the checklist

- **Your manifest** (`.limitless-project.py`): your notebook IDs, your Pinecone index, which optional checks run.
- **Your CLAUDE.md**: every `[YOUR ...]` marker filled; edit it whenever a convention stops working.
- **Your wiki**: your overview, your sources, your entities. The starter pages are yours to change.
- **Your lessons**: add your own anti-patterns after the twelve starters. Never renumber.
- **Keep as-is**: the tools and skills (update them with `git pull + install.sh`), the first-action banner, the end-of-session checklist.

Sharing a vault with teammates (optional)

A vault can be one team's shared record: everyone reads everything and adds freely, but each person's writing can only be changed by that person. Anyone else who wants a change sends the owner a suggestion; the owner approves it and makes the change. Skip this for a vault only you write to.

1. Turn on the rule. Copy `~/LimitlessStack/templates/authors.example.json` to `.authors.json` at the vault root and list each person's GitHub login with every email they commit with. In `.limitless-project.py` set `VAULT_OWNER = "<your - login>"`: the vault's upkeep checks then run on your Roll Call, and each teammate's Roll Call shows only their own machine, sign-ins and task file. Commit both.

2. Each person, on their own computer: clone the vault, then run `bash tools/install-git-hooks.sh`. From then on a commit that changes someone else's lines, their `members/<login>/` folder or their `wiki/my-tasks/<login>.md` is refused, and the message says how to send a suggestion instead.

3. Personal rules. Each person keeps their own agent instructions in `members/<login>/CLAUDE.md`, which everyone can read.

4. Optional backstop. Copy `templates/github/authorship-safety-net.yml` to `.github/workflows/`. Each night it checks the day's commits and opens an issue for anyone whose writing was changed without the check, so they decide. Run it once by hand to prove it works.

Check: `bash tools/test-authorship-guard.sh` ends with "0 failed".

09

Using Grok Bot

xAI's always-on agents, working on your own Mac

Grok Bot runs on xAI's own cloud computer. Your stack lives on your Mac: your vault, your Keychain, your NotebookLM sign-in. So Grok Bot has to run the stack's commands on your Mac, and it needs the stack's rules handed to it. Two steps make that work.

1 Let it run commands on your Mac

In the Grok Bot desktop app on your Mac, open **Grok Bot, Settings, Computer** and set **Execution on this computer** to **Ask every time** (the default is *Never allow*). Grok Bot then asks before each command: *View the command* shows it, **Allow once** runs just that one, the X denies it. Keep *Always allow* off until you trust the routine.

Type passwords and sign-in codes yourself, never into the chat. Keep your keys in your Mac's Keychain, not on the Bot's cloud computer, which every Bot on your account can use.

2 Give it the stack's rules as a skill

Don't count on Grok Bot reading `AGENTS.md`, `CLAUDE.md` or the Claude skills by itself. Grok Bot makes private skills for you: paste this to your Bot, fill in the two `<...>` values, and ask it to *save this as a private skill, exactly as written*. It then shows under Marketplace, Your plugins. Each command Grok Bot runs starts fresh in your home folder, so the skill uses full paths.

```
Name: Limitless Stack session
When to use: before any work in my vault at <vault path on my Mac>.
Access: Execution on this computer (my Mac).
Rule: every command starts in my home folder, so always use full
paths, with the vault path in quotes.
Steps:
1. Read <vault path>/AGENTS.md, then <vault path>/CLAUDE.md in
   full, then <vault path>/members/<my-github-login>/CLAUDE.md if
   it exists, and follow them. Where they say Skill(...) or
   Desktop Commander, run the same command directly on my Mac.
2. Run Roll Call as one command:
   bash '<vault path>/tools/limitless-preflight.sh'
   0 = READY. 1 = WARN, tell me. 2 = BLOCK, stop and tell me.
3. Do the task.
4. Before finishing: the end-of-session checklist in CLAUDE.md.
Validate: Roll Call printed a verdict; the checklist is done.
Return: the Roll Call verdict, what changed, anything left open.
Needs my approval: every commit or push, any write to a repo,
and any command outside the vault.
```

Check: in a chat, say *"Run the Limitless Stack session skill, Roll Call only."* It asks to read the files and run Roll Call on your Mac, then reports READY, WARN or BLOCK.

Sharing a vault? The same skill works in a teammate's copy of a shared vault. Their Roll Call shows only their own machine, sign-ins and task file (section 08).

10 Troubleshooting

What Roll Call and the tools tell you, and what to do

You see	Do this
BLOCK: NotebookLM auth failing	Run notebooklm login on your computer, then notebooklm auth check --test.
"No Pinecone index configured"	Add PINECONE = {"index": ""} to .limitless-project.py (step 7).
Notebook coverage warning	Every notebook in your Google account must be in your manifest: a route, the default, the reminder, or "ignored".
verify_failed above 0	The upload didn't land. Delete that source in the notebook and add the file again; don't just re-run.
A refresh seemed to time out	Check the notebook before retrying — large files take minutes to index.
Tools out of sync with LimitlessStack	You edited a tool in one place. Copy it to the other, or re-run install.sh.
overview.md placeholder warning	Write your overview and give it real dates (step 5).
Your agent can't run notebooklm	It must run on your own computer, where you signed in — not inside a cloud sandbox.
Grok Bot can't reach your Mac	Settings, Computer: set Execution on this computer to Ask every time (not Never allow).
Grok Bot can't find the vault	Its commands start in your home folder: use the full vault path in quotes, as the skill does.

The Limitless Stack is one integrated system: the value compounds across all seven tools, every session, every source and every lesson. The stack you run on day 90 won't be the one you started with — that's the point.